SINOFETCH

Database Schema — Intial Architecture & Review

Version 1.0

Prepared and Revised By: Chamindu Vidyarathne (RM)

Database Schema

2.1 organization

Represents a business account (Sinofetch, or future SaaS customers). Linked to Supabase Auth.

Column	Type	Constraints	Notes
org_id	UUID	PK, DEFAULT gen_random_uuid()	Primary key
owner_user_id	UUID	FK → auth.users(id), NOT NULL	Supabase authenticated user who owns this org
org_name	TEXT	NOT NULL	Display name e.g. 'Sinofetch Ltd'
email	TEXT	UNIQUE, NOT NULL	Primary contact / billing email
is_active	BOOLEAN	DEFAULT TRUE	Soft-disable an org without deleting data
created_at	TIMESTAMPTZ	DEFAULT now()	Row creation timestamp
updated_at	TIMESTAMPTZ	DEFAULT now()	Auto-updated via moddatetime trigger

2.2 persona

Defines an AI chatbot's identity, behaviour, and model configuration. One org can have multiple personas (e.g. Sales bot, Support bot). For the initial stage, Only one persona per one org.

Column	Type	Constraints	Notes
persona_id	UUID	PK, DEFAULT gen_random_uuid()	Primary key
org_id	UUID	FK → organization(org_id), NOT NULL	Owning organisation

Column	Type	Constraints	Notes
name	TEXT	NOT NULL	Display name e.g. 'Sinofetch Assistant'
system_prompt	TEXT	NOT NULL	Full system prompt injected into AI context
greeting_message	TEXT	—	First message shown when chat opens
fallback_message	TEXT	—	Message when AI cannot answer
ai_provider	TEXT	DEFAULT 'openai'	AI backend: openai | anthropic
model_name	TEXT	DEFAULT 'gpt-4o'	Model identifier e.g. gpt-4o, claude-sonnet-4-5
temperature	NUMERIC(3,2)	DEFAULT 0.7 CHECK (0..1)	Response creativity (0 = precise, 1 = creative)
max_tokens	INTEGER	DEFAULT 1024	Max tokens per AI response
is_active	BOOLEAN	DEFAULT TRUE	Disable persona without deleting
created_at	TIMESTAMPTZ	DEFAULT now()	Row creation timestamp
updated_at	TIMESTAMPTZ	DEFAULT now()	Auto-updated via trigger

2.3 consumer

A website visitor who engages with the chat bubble. Contact details captured here feed the leads pipeline.

Column	Type	Constraints	Notes
consumer_id	UUID	PK, DEFAULT gen_random_uuid()	Primary key
persona_id	UUID	FK → persona(persona_id), NOT NULL	Which persona the consumer interacted with
org_id	UUID	FK → organization(org_id), NOT NULL	Denormalized for fast admin queries
name	TEXT	—	Provided by consumer (optional until captured)
email	TEXT	—	Provided by consumer
phone_number	TEXT	—	Provided by consumer
fingerprint	TEXT	—	Browser fingerprint / anonymous ID for returning visitors
ip_address	INET	—	IP at first interaction (GDPR — mask if required)
created_at	TIMESTAMPTZ	DEFAULT now()	First seen
updated_at	TIMESTAMPTZ	DEFAULT now()	Last data update

Index recommendation
CREATE INDEX idx_consumer_email ON consumer(email, persona_id); This prevents duplicates from being created for returning visitors contacting the same bot.

2.4 chat_session

A single conversation thread. Tracks full lifecycle from open to close, and records whether a lead was captured.

Column	Type	Constraints	Notes
session_id	UUID	PK, DEFAULT gen_random_uuid()	Primary key
consumer_id	UUID	FK → consumer(consumer_id), NOT NULL	Who started the session
persona_id	UUID	FK → persona(persona_id), NOT NULL	Which persona handled it
org_id	UUID	FK → organization(org_id), NOT NULL	Denormalized for fast admin filtering
status	TEXT	DEFAULT 'active'	active | closed | abandoned
lead_captured	BOOLEAN	DEFAULT FALSE	True once consumer contact info is confirmed
started_at	TIMESTAMPTZ	DEFAULT now()	Session open time
ended_at	TIMESTAMPTZ	—	NULL while active; set on close/abandon
created_at	TIMESTAMPTZ	DEFAULT now()	Row creation timestamp
updated_at	TIMESTAMPTZ	DEFAULT now()	Auto-updated via trigger

2.5 message

Individual messages within a session. Uses an ENUM for source and a sequence number for deterministic ordering.

Column	Type	Constraints	Notes
msg_id	UUID	PK, DEFAULT gen_random_uuid()	Primary key
session_id	UUID	FK → chat_session(session_id), NOT NULL	Parent session

Column	Type	Constraints	Notes
consumer_id	UUID	FK → consumer(consumer_id)	NULL for assistant/system messages
sequence_no	INTEGER	NOT NULL	1-based order within session; deterministic sort
msg_source	TEXT	CHECK IN ('user','assistant','system')	Who produced this message
content	TEXT	NOT NULL	Full message text
tokens_used	INTEGER	—	AI tokens consumed (for billing/monitoring)
is_deleted	BOOLEAN	DEFAULT FALSE	Soft delete — preserves audit trail
created_at	TIMESTAMPTZ	DEFAULT now()	Message timestamp

Ordering strategy
Always sort by sequence_no, then created_at as a tiebreaker. Set sequence_no in your API when saving: SELECT COALESCE(MAX(sequence_no), 0) + 1 FROM message WHERE session_id = $1

2.6 qualified_lead

Captures a confirmed lead from a conversation. A consumer may have multiple sessions but only one qualified lead record per persona. This feeds the admin Leads dashboard.

Column	Type	Constraints	Notes
lead_id	UUID	PK, DEFAULT gen_random_uuid()	Primary key
consumer_id	UUID	FK → consumer(consumer_id), NOT NULL	The captured consumer
session_id	UUID	FK → chat_session(session_id)	Session in which lead was captured
org_id	UUID	FK → organization(org_id), NOT NULL	Owning org for filtering
name	TEXT	—	Snapshot at capture time (denormalized)
email	TEXT	—	Snapshot at capture time
phone_number	TEXT	—	Snapshot at capture time
notes	TEXT	—	Admin notes / follow-up comments

Column	Type	Constraints	Notes
status	TEXT	DEFAULT 'new'	new | contacted | qualified | closed
captured_at	TIMESTAMPTZ	DEFAULT now()	When lead was created
updated_at	TIMESTAMPTZ	DEFAULT now()	Auto-updated via trigger

Why snapshot contact fields here?
Consumer details might be updated later. The lead record captures the data at the moment of qualification so admin always sees what was collected during that conversation.

3. Entity Relationship Summary

Parent Table		Child Table	Cardinality
organization	→	persona	1 org, many personas
organization	→	consumer	1 org, many consumers
organization	→	chat_session	1 org, many sessions
organization	→	qualified_lead	1 org, many leads
persona	→	consumer	1 persona, many consumers
persona	→	chat_session	1 persona, many sessions
consumer	→	chat_session	1 consumer, many sessions
consumer	→	qualified_lead	1 consumer, 1+ leads
chat_session	→	message	1 session, many messages
chat_session	→	qualified_lead	1 session, 0–1 leads

4. Supabase Implementation Notes

4.1 Row Level Security (RLS)

Enable RLS on all tables. Staff only ever see their own org's data.

Critical RLS policy pattern (apply to every table)

```
CREATE POLICY org_isolation ON chat_session   FOR ALL USING (org_id = (SELECT
org_id FROM organization WHERE owner_user_id = auth.uid()));
```

4.2 Auto-update Trigger

Enable the moddatetime extension and attach it to every table that has an updated_at column.

One-time setup in Supabase SQL editor

```
CREATE EXTENSION IF NOT EXISTS moddatetime;  CREATE TRIGGER handle_updated_at
BEFORE UPDATE ON organization   FOR EACH ROW   EXECUTE PROCEDURE
moddatetime(updated_at);  -- Repeat for: persona, consumer, chat_session,
qualified lead
```

4.3 Recommended Indexes

- consumer(email, persona_id) — deduplicate returning visitors
- chat_session(org_id, status) — admin dashboard conversation filter
- chat_session(consumer_id) — load all sessions for a consumer
- message(session_id, sequence_no) — fast in-order message fetch
- qualified_lead(org_id, status) — leads page filter & sort
- qualified_lead(consumer_id) — link lead back to consumer

5. Build Order

Follow this order to avoid migration problems and enable incremental testing.

Step 1	**Create organization + auth.users link**	Core tenant model — everything else depends on this
Step 2	**Create persona table with explicit columns**	Unblock chat UI and API config endpoints
Step 3	**Create consumer + chat_session**	Enable chat sessions to be opened and tracked
Step 4	**Create message with sequence_no**	Full chat flow now functional end-to-end
Step 5	**Create qualified_lead + RLS policies**	Admin leads dashboard can be built
Step 6	**Enable Realtime on message + chat_session**	Admin live monitoring; streaming responses

Step 7	**Add moddatetime triggers + all indexes**	Performance and data integrity hardening